Auditing Closed-Loop Learning in Recurrent Neural Networks: Reproduction, Robustness, and Generalization

Anonymous authors
Paper under double-blind review

Abstract

Recurrent neural networks are often used as mechanistic models of learning and control, but closed-loop training creates reproducibility challenges because the model's actions alter future inputs. We conduct a claim-level reproducibility study of Ger and Barak's analysis of closed-loop RNN learning dynamics. Rather than asking only whether original figures can be regenerated, we test whether the paper's main claims survive independent implementation, seed variation, optimizer and hyperparameter perturbations, coupled-system stability diagnostics, and transfer to nonlinear or path-integration-style settings. Across [**TBD: insert number of completed paired runs**], we find [**TBD: insert principal empirical result after full audit**]. Our results are intended to produce generalizable lessons for closed-loop RNN studies: deployed closed-loop loss, paired-seed comparisons, algorithmic stage criteria, coupled-system spectra, feedback strength, rollout horizon, and seed-level failure rates should be reported whenever papers make claims about recurrent learning dynamics in interactive environments.

1 Introduction

Closed-loop recurrent neural networks occupy an important middle ground between supervised sequence modeling, control, and computational neuroscience. In a closed-loop environment, an RNN's output changes the future inputs it receives. This feedback makes the training problem qualitatively different from open-loop imitation or teacher-forced supervised learning, where future inputs are independent of the learner's deployed actions. Ger and Barak (Ger and Barak, 2025) recently argued that this distinction produces learning dynamics that are fundamentally different from those of otherwise identical open-loop RNNs. Their paper is valuable because it makes a mechanistic claim rather than only a performance claim: the coupled agent-environment system, not the RNN alone, determines important learning transitions.

This paper is not a figure-level reproduction study. Instead, we use Ger and Barak's closed-loop RNN learning framework as a test case for claim-level reproducibility in recurrent control systems. We ask which conclusions survive independent implementation, seed variation, optimizer and hyperparameter perturbations, coupled-system stability analysis, and transfer to nonlinear or path-integration-style settings. This framing matters for TMLR-style reproducibility: a rerun of a public artifact can be educational, but it is only broadly useful when it surfaces reusable lessons about when claims are reliable, when they are protocol-sensitive, and what future papers should report.

Our study makes four contributions. First, we provide an independent, config-driven reproduction of the central open-loop/closed-loop learning comparisons in Ger and Barak. Second, we convert qualitative claims about learning stages and stability transitions into quantitative, seed-level tests with confidence intervals. Third, we measure robustness under perturbations to optimizer, initialization, feedback strength, noise, and rollout horizon, identifying which conclusions are stable and which are protocol-sensitive. Fourth, we distill an audit checklist for future closed-loop RNN studies, including paired-seed comparisons, deployed closed-loop loss, stage criteria, coupled-system stability diagnostics, and failure-rate reporting.

2 Related Work

Task-trained RNNs in computational neuroscience. Task-trained RNNs are widely used as mechanistic models because they can solve cognitive and motor tasks while exposing internal dynamics for analysis. Early and influential work showed that high-dimensional trained RNNs can implement low-dimensional dynamical mechanisms (Sussillo and Barak, 2013), and that recurrent dynamics can account for context-dependent computation in prefrontal cortex (Mante et al., 2013). This line of work has become a general modeling strategy in systems neuroscience, where RNNs serve as hypothesis-generating models rather than only predictive tools (Barak, 2017). At the same time, large-scale analyses of trained recurrent networks show that architecture and training details can change representational geometry and dynamics even when task performance is similar (Maheswaranathan et al., 2019). This motivates our claim-level framing: reproducing a task curve is not enough if the mechanistic conclusion depends on training protocol or model class.

Dynamical-systems analysis of RNNs. The original paper sits in a tradition of reverse-engineering trained RNNs through fixed points, low-dimensional projections, eigenvalues, and linearized dynamics. Sussillo and Barak's low-dimensional analysis made the case that trained recurrent networks can be understood through dynamical-systems tools (Sussillo and Barak, 2013), and later work used related analyses to compare recurrent dynamics across large populations of networks (Maheswaranathan et al., 2019). Our audit differs in emphasis. We do not only ask whether a trained RNN has interpretable internal dynamics; we ask whether the relevant dynamical object is the RNN alone or the coupled agent-environment system. This distinction is central in closed-loop settings because the environment transforms actions into future observations.

Open-loop training, closed-loop deployment, and distribution shift. The mismatch between teacher-forced training and deployed rollouts is well known in sequence prediction and imitation learning. Scheduled sampling was proposed to reduce the discrepancy between training on ground-truth previous tokens and testing on model-generated tokens (Bengio et al., 2015). In imitation learning, DAgger and related reductions explicitly address the fact that a learner's actions change the future states it visits, invalidating i.i.d. supervised-learning assumptions (Ross et al., 2011). Closed-loop RNN learning has an analogous but mechanistically richer mismatch: open-loop imitation can minimize teacher-forced action error while still failing under deployed closed-loop rollouts. This connection motivates our insistence on reporting deployed closed-loop loss, not only open-loop or imitation loss.

Reproducibility and robustness in machine learning. Our audit follows the broader shift from artifact reruns to structured reproducibility studies. The NeurIPS reproducibility program emphasized clear artifacts, documented environments, and reproducibility checklists (Pineau et al., 2021). In deep reinforcement learning, Henderson et al. showed that algorithmic conclusions can be sensitive to seeds, evaluation protocol, and implementation details (Henderson et al., 2018). Closed-loop RNN studies share many of these risks because training outcomes can depend on random initialization, rollout horizon, optimizer, and environment coupling. The present paper therefore treats Ger and Barak's work as a case study for a reusable audit protocol: paired seeds, stress tests, coupled-system diagnostics, and reporting lessons.

3 Original Claims and Audit Questions

We organize the study around five claims. C1 is the central divergence claim: identical RNNs trained open-loop and closed-loop follow different learning trajectories and differ under closed-loop deployment. C2 is the stage claim: closed-loop learning passes through distinct learning stages. C3 is the mechanistic stability claim: progress depends on stability of the coupled agent-environment system. C4 is the protocol robustness claim: short-term policy improvement and long-term stability interact with optimizer, horizon, feedback strength, and other choices. C5 is the generalization claim: similar dynamics appear beyond the simplest double-integrator setting.

For each claim, we define three evidence levels. A claim is *reproduced* when the direction of the effect and the relevant diagnostic hold under the original protocol for most paired seeds with confidence intervals excluding negligible effects. A claim is *partially reproduced* when the direction appears under the original protocol but is unstable across seeds, sensitive to plausible perturbations, or only weakly supported by the diagnostic. A

Table 1: Closed-loop RNN reproducibility audit protocol. The protocol is intentionally lightweight: each row defines a required audit step, its minimum implementation, and the output needed to support claim-level conclusions.

Audit step	**Minimum requirement**	**Recorded output**
Paired training	Clone the same initialization into open-loop and closed-loop learners; keep architecture and evaluation budget fixed.	Per-seed train loss, deployed closed-loop test loss, peak deployed loss, final loss, and effective-gain trajectory for both learners.
Seed-level uncertainty	Run enough seeds to estimate direction agreement and confidence intervals; report seed count explicitly.	Mean, 95 percent bootstrap CI, fraction of seeds supporting each claim, and seed-level failure/recovery rates.
Stage detection	Predefine a detector using log-loss derivative, minimum plateau duration, and optional stability crossing.	Stage labels, plateau duration, plateau loss, transition epochs, and frequency of detected stages across seeds/settings.
Coupled diagnostics	Compute RNN-only and coupled agent-environment spectra whenever the system can be linearized or locally linearized.	Spectral radius time series, stability-crossing epoch, plateau-exit gap, and predictive correlation with recovery/failure.
Robustness perturbations	Change factors that alter optimization or environment coupling, such as optimizer, initialization scale, feedback strength, noise, horizon, control penalty, and clipping.	Claim support under each perturbation, effect-size changes, and perturbation-level failure rates.
Generalization check	Test at least one architecture or task variant that preserves action-dependent inputs and one that changes the recurrent/control structure.	Claim support by variant and a boundary-condition statement explaining when the phenomenon does or does not transfer.
Claim decision	Classify each claim as reproduced, partially reproduced, or not reproduced using the same criteria across settings.	Claim-level matrix linking evidence type, original setting, robustness, generalization, and actionable lesson.

claim is *not reproduced* when the effect is absent, reverses, or depends on hand-picked visual criteria. These labels are intended to avoid the ambiguous statement that a result "matches the original" without specifying the evidence.

4 Closed-Loop RNN Reproducibility Audit Protocol

We specify a modest audit protocol rather than proposing a new benchmark. The protocol is a recipe for testing claims about closed-loop RNN learning dynamics in any recurrent control setting where actions can affect future observations. Its inputs are: a set of original claims, a task/environment, an open-loop training procedure, a closed-loop training procedure, a seed set, a small set of protocol perturbations, and at least one held-out architecture or task variant. Its outputs are: per-seed deployed closed-loop metrics, stage labels, stability diagnostics, perturbation summaries, generalization summaries, and a claim-level reproducibility matrix. Table 1 gives the concrete specification used in this paper.

The protocol deliberately separates *measurement* from *interpretation*. Measurement consists of paired runs, deployed losses, spectra, stage labels, and perturbation outcomes. Interpretation happens only after aggregating seed-level outputs into claim-level decisions. This separation is important because closed-loop RNN papers often make qualitative statements about stages or mechanisms from a small number of curves; our protocol requires those statements to be tied to explicit detectors, uncertainty estimates, and failure rates.

4.1 Paired open-loop and closed-loop training

For each seed, we initialize one recurrent controller and clone its weights. The closed-loop model is trained by rolling the controller in the environment and minimizing deployed state cost plus a control penalty. The

open-loop model is trained as a student to imitate a teacher trajectory generated by the trained closed-loop controller. Both models are then evaluated under closed-loop deployment. The key dependent variables are final deployed loss, peak deployed loss, effective feedback-gain trajectory, failure or recovery status, and time to stable performance.

4.2 Stage detection

The original paper describes closed-loop learning stages visually. We operationalize this claim. Stage 1 is early rapid improvement, Stage 2 is a plateau or slow-progress phase, and Stage 3 is post-transition improvement. The detector uses log-loss derivatives, a minimum plateau duration, and optionally a coupled spectral-radius crossing. This makes stage claims falsifiable: a stage either appears consistently across seeds and perturbations or it does not.

4.3 Coupled-system stability diagnostics

For linearized settings, we compute the spectral radius of the RNN recurrent matrix and of the coupled agent-environment transition matrix. The audit asks whether the coupled-system spectral radius predicts plateau exit, instability, or recovery better than the RNN-only spectral radius. This test directly targets the mechanistic contribution of the original paper: if closed-loop dynamics are genuinely about the agent-environment loop, the coupled diagnostic should be more predictive than an isolated RNN diagnostic.

4.4 Robustness and generalization

The robustness sweep changes learning rate, initialization scale, feedback strength, episode length, observation noise, control penalty, optimizer, and gradient clipping. The generalization sweep tests nonlinear RNNs, GRUs, low-rank RNNs, a tracking-control variant, and a ring/path-integration-style task. We do not treat every extension as a new benchmark. Instead, each extension asks whether the closed-loop/open-loop distinction persists when the task still has action-dependent inputs and partial observability.

5 Result 1: Direct Reproduction of the Core Divergence

[TBD: Fill after full audit. Report number of paired seeds, mean and 95 percent bootstrap confidence interval for final closed-loop deployed loss, final open-loop deployed loss, gain-trajectory distance, fraction of seeds showing open-loop deployed-loss spike, and fraction showing a closed-loop plateau.]

The core result should be interpreted as claim-level evidence rather than a visual overlay of curves. The minimal evidence for C1 is that paired models with the same initialization systematically differ in deployed closed-loop performance or effective-gain trajectories. If teacher-forced open-loop loss decreases while deployed closed-loop loss remains high or spikes, this supports the reporting lesson that open-loop training loss is insufficient for interactive systems.

6 Result 2: Which Claims are Robust?

[TBD: Fill after robustness sweep. Summarize perturbation effects for learning rate, initialization, feedback strength, horizon, noise, optimizer, control penalty, and clipping. Include failure rates and confidence intervals.]

This section should emphasize protocol sensitivity, not just success. For example, if the open-loop/closed-loop divergence is robust to initialization but weaker under Adam or high feedback gain, the contribution is the boundary condition. A robust claim is one that survives plausible protocol choices; a useful reproducibility report identifies both the stable conclusions and the fragile ones.

Table 2: Claim-level reproducibility matrix. Result cells marked TBD will be populated from the completed full audit.

Claim	Evidence type	Original protocol	Robustness sweep	Generalization	Lesson
C1: open/closed divergence	Paired seeds, effect size, CI, deployed loss	[**TBD: reproduced/partial/fail**]	[**TBD: robust except under X**]	[**TBD: holds for Y, fails for Z**]	Report deployed closed-loop loss, not only teacher-forced loss.
C2: stage structure	Stage detector, plateau duration, transition variability	[**TBD: reproduced/partial/fail**]	[**TBD: sensitive to X**]	[**TBD: weak/strong in Y**]	Stage claims require algorithmic criteria, not visual identification.
C3: coupled stability	Spectral radius vs plateau exit or failure	[**TBD: reproduced/partial/fail**]	[**TBD: robust under X**]	[**TBD: partial/full**]	Report coupled-system spectrum, not only RNN-only spectrum.
C4: protocol robustness	Hyperparameter and optimizer perturbations	N/A	[**TBD: mixed/stable/fragile**]	N/A	Include seed and hyperparameter sensitivity.
C5: broader generalization	Architecture and task extensions	N/A	N/A	[**TBD: partial/full/fail**]	Closed-loop effects depend on environment coupling and partial observability.

7 Result 3: Which Diagnostics are Predictive?

[TBD: Fill after spectral analysis. Compare correlations between plateau exit/failure and coupled-system spectral radius versus RNN-only spectral radius. Report timing gap between stability crossing and plateau exit.]

The central mechanistic question is whether closed-loop progress is better explained by the coupled system than by the RNN alone. A positive result would support the original interpretation and lead to a reporting recommendation: closed-loop RNN papers should report coupled-system stability metrics whenever possible. A negative or mixed result would also be informative, because it would delimit when spectral-radius analysis is useful.

8 Result 4: Does the Phenomenon Generalize?

[TBD: Fill after generalization sweep. Report which architecture/task variants preserve C1–C3, and where the claims weaken or fail.]

We interpret generalization through environment structure. The expected lesson is not that all architectures must reproduce the same curves. Instead, closed-loop/open-loop divergence should be strongest when actions affect future observations and when partial observability makes recurrent state useful. If a task removes these ingredients, failure to reproduce the dynamics is a boundary condition rather than an implementation failure.

9 Claim-Level Reproducibility Matrix

Table 2 is the main summary of the paper. It is designed to make the audit more than a class-report rerun: each original claim is attached to evidence, robustness, generalization, and an actionable lesson.

10 Actionable Lessons for Closed-Loop RNN Studies

The goal of the audit is to produce reusable reporting practices. We propose the following checklist for closed-loop RNN papers.

Report deployed closed-loop performance, not only open-loop or teacher-forced loss. The central risk is that a model can imitate a teacher under fixed inputs but fail when its own actions determine future inputs.

Use paired-seed comparisons when contrasting open-loop and closed-loop training. The clean comparison uses the same initialization, architecture, optimizer budget, and evaluation rollouts while changing only the training regime.

Define learning stages algorithmically. If a paper claims stages, it should specify transition criteria, minimum plateau duration, and how often each stage appears across seeds.

Analyze the coupled agent-environment system. RNN-only spectra can miss stability changes introduced by the environment. Coupled diagnostics should be reported whenever a mechanistic stability claim is made.

Report feedback strength and rollout horizon. These parameters determine how strongly actions affect future observations and can control whether the closed-loop phenomenon appears.

Report failure rates, not just average curves. Means can hide unstable runs, recovery failures, or seed-dependent plateaus.

Include at least one stress test. Optimizer, initialization, noise, feedback strength, and horizon perturbations are inexpensive compared with the interpretive cost of an untested mechanistic claim.

11 Discussion

This project treats Ger and Barak's framework as a case study in reproducibility for interactive recurrent systems. The expected contribution is not that every curve exactly matches the original paper. Rather, the contribution is to identify which claims are robust, which require careful protocol choices, and which diagnostics future closed-loop RNN papers should report. This is especially important for computational neuroscience, where RNNs are often used as mechanistic models and where closed-loop behavior is closer to biological learning than fixed-dataset supervision.

12 Limitations

[**TBD: Update after experiments.**] The audit does not cover every RNN architecture, every control task, or full reinforcement-learning algorithms. The path-integration extension is intentionally simple. Some original motor-control details may not be reproduced exactly. Finally, the audit protocol emphasizes practical reproducibility diagnostics; it does not prove that the original theoretical explanation is necessary or unique.

13 Artifact

The accompanying repository provides a config-driven implementation with smoke and full-run modes, raw CSV outputs, claim tables, and figure-generation scripts. The artifact is organized around the audit protocol rather than notebook-only reproduction: each experiment records the config, seed, device, metrics, and per-epoch time series. Full-run results can be regenerated with `bash scripts/run_full_audit.sh`; a lightweight validation run is available with `bash scripts/run_smoke.sh`.

References

Yoav Ger and Omri Barak. Learning dynamics of RNNs in closed-loop environments. In *Advances in Neural Information Processing Systems*, poster, 2025. OpenReview: `https://openreview.net/forum?`

id=P63bBMXCIH.

Omri Barak. Recurrent neural networks as versatile tools of neuroscience research. *Current Opinion in Neurobiology*, 46:1–6, 2017.

Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems*, 2015.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018.

Niru Maheswaranathan, Alex H. Williams, Matthew D. Golub, Surya Ganguli, and David Sussillo. Universality and individuality in neural dynamics across large populations of recurrent networks. In *Advances in Neural Information Processing Systems*, 2019.

Valerio Mante, David Sussillo, Krishna V. Shenoy, and William T. Newsome. Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503:78–84, 2013.

Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Lariviere, Alina Beygelzimer, Florence d'Alché-Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research: a report from the NeurIPS 2019 reproducibility program. *Journal of Machine Learning Research*, 22(164):1–20, 2021.

Stéphane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*, pages 627–635, 2011.

David Sussillo and Omri Barak. Opening the black box: low-dimensional dynamics in high-dimensional recurrent neural networks. *Neural Computation*, 25(3):626–649, 2013.

Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.